Table 1: Dictionary for interpreting behavior control and modification

Node	Description	Behavior Template
Sequence	Execute a series of child nodes in order from left to right.	Execute the following actions ([Actions]) in order and stop the sequence if any action fails.
SequenceWithMemory, SequenceStar	Execute a series of child nodes in order, from left to right, remembering which ones have already succeeded.	Execute the following actions ([Actions]) in sequence. If an action fails, stop the sequence; on the next execution, resume from the failed action without repeating the actions that already succeeded.
Delay	Wait for a given duration before executing the associated child node.	Wait for the specified delay, then execute [Action] and return its result.
PipelineSequence	Execute a series of child nodes in order from left to right and advance on running or success.	Execute the following actions ([Actions]) in order and stop the sequence if any action fails. While an action is running or has succeeded, go to the next action.
ConditionalSequence	Execute a series of child nodes in order from left to right and re-check a condition for each node.	Execute the following actions ([Actions]) in order while checking a condition for each action. If [Condition] move to the next action. Else if [!Condition], end the sequence.
WhileDoElse	Execute a node while a condition is true else execute and alternative node if that same condition fails.	While [Condition], do [Action]. If [!Condition], execute the alternative.
Loop	Repeatedly execute a child node for a specified number of iterations, until a specific status is returned or forever.	Execute [Action] continuously until it fails or succeeds. Else keep executing [Action] forever.
Fallback	Execute child nodes sequentially from left to right, until one succeeds or all fail.	Execute the following actions ([Actions]) consecutively. If any action ([Action]) fails, move to the next action until an action succeeds. If no action succeeds, the execution fails.
FallbackStar	Execute child nodes sequentially from left to right, until one succeeds. If all fail retry again from the beginning.	Execute the following actions ([Actions]) consecutively. If any action ([Action]) fails, move to the next action until an action succeeds. If no action succeeds, start all over again until an action succeeds.
ForceSuccess	Force child node to succeed regardless of the conditions.	Execute [Action] and make sure it always succeeds.
ForceFailure	Force child node to fail regardless of the conditions.	Execute [Action] and make sure it always fails.
RepeatUntilSuccess	Repeatedly execute child node until its successful.	Continuously execute [Action] until it succeeds.
Retry	Re-execute a child node for either a fixed number of times or until its successful.	Execute [Action]. If it fails, re-execute it either for a specified number of times or until it succeeds.
ReactiveSequence	Execute a series of child nodes in sequence. While a node is running, re-evaluate previous child nodes.	Execute the following actions ([Actions]) in sequence. If [Action] fails, stop execution. While [Action] is still running, re-evaluate previous actions.
ReactiveFallback	Execute a series of child nodes in order from left to right. While a later node is running, restart evaluation from node 0.	Execute the following actions [Actions] in order. If [Action] succeeds, stop execution. Else keep executing and re-evaluating previous actions whenever an action is running.
Repeat	Execute a set of child nodes multiple times.	Execute [Action] for a set number of times or infinitely.
Inverter	Negate a the execution outcome of a child node.	Execute inverted [Action]. If [Action] succeeds, reverse it to fail. Else if [Action] fails, reverse it to succeed.
KeepRunningUntilFailure	Keep executing child node as long as it does not fail.	Execute [Action]. If [Action] succeeds, restart it. Else if [Action] fails, stop looping and continue execution.
Switch (Switch2–Switch6)	Execute child node based on blackboard value.	Read blackboard value and compare it against a set of cases. If a case matches the blackboard value, execute corresponding [Action]. Else if no case matches, execute default [Action].
Parallel, ParallelNode	Execute multiple child nodes simultaneously, ensuring all are completed successfully.	Execute the following actions ([Actions]) simultaneously. If either a success threshold is reached or all actions succeed, the execution was successful. Else if any action fails, the execution fails.
BlackboardCheckValue (Int, Double, String)	Check whether a value stored on the blackboard matches the expected value.	Find a value on the blackboard and check if it matches a given rule.
RetryUntilSuccessful, RetryUntilSuccesful	Continuously try to re-execute a child node until it succeeds, up to a bounded number of attempts.	Execute [Action]. If it fails, re-execute it up to [port:num_attempts] attempts in total. If it has still not succeeded, the execution fails. If [port:num_attempts] is -1, keep retrying indefinitely.
RecoveryNode	Try to run the primary child node. If it fails, run the next child node to recover and try the primary child node again, for a bounded number of retries.	Execute main [Action]. If it fails, execute recovery action ([Action]), then retry main [Action]. Repeat this recovery-and-retry cycle up to [port:number_of_retries] times. If main [Action] still has not succeeded after that many retries, or if recovery action ([Action]) fails, stop execution.
Timeout	Execute child node, but only for a maximum duration.	If [Action] fails to execute within specified time limit, interrupt the execution and stop it.
ManualSelector	Let a human operator choose at runtime which child node to execute.	Ask the operator which of the following actions ([Actions]) to execute, then execute the selected [Action] and return its result.
RoundRobin	Execute child nodes one at a time in rotating order, resuming from the child after the one last executed on the previous tick.	Execute the next action in the rotation ([Actions]). If [Action] succeeds, stop execution and continue from the following action next time. If [Action] fails, move to the next action. If all actions fail in one full rotation, the execution fails.
IfThenElse	Execute a condition child and, depending on its result, execute either the second or the third child node.	If [Condition], execute [Action]. Else if [!Condition], execute the alternative [Action]. If no alternative is given, the execution fails.
RateController	Execute a child node at most at a given frequency.	Execute [Action] at a fixed rate. If the specified period has not yet elapsed, wait before executing [Action] again.
GoalUpdater	Update the current goal before executing a child node.	Check for a new goal. If a new goal is available, replace the current goal with it, then execute [Action].
DistanceController	Execute a child node each time the robot has travelled a given distance.	Execute [Action] once the robot has moved the specified distance since the last execution. Else do not re-execute [Action].
SpeedController	Execute a child node at a frequency that scales with the robot's speed.	Execute [Action] more often when the robot moves faster and less often when it moves slower, within a minimum and maximum rate.
SubTree	Execute another behavior tree referenced by its ID.	Execute the subtree [Subtree ID] and return its result as if it were a single [Action].

Placeholders: **[Action]**: a child node name; if it is a standard ROS action, controller or decorator, refer to it as *ROS action [name]*, else as *custom action [name]*. **[Actions]**: a series of child node names, listed by name. **[Condition]**: a check that returns success, rendered as *[condition ID] is true*. **[!Condition]**: a check that returns failure, rendered as *[condition ID] is false*. **[Subtree ID]**: the ID of a referenced behavior tree, rendered as *subtree [ID]*. **[port:*name*]**: the value of the node's *name* port as written in the XML (e.g. `num_attempts`). An integer is inserted as written, e.g. `num_attempts="5"` gives *up to 5 attempts*. `-1` means "no limit" and is written that way: *indefinitely* for attempt, retry and cycle counts, *all children* for a `Parallel` threshold. A blackboard reference `{key}` becomes *a value read from [key] at runtime* and is never left out. If the attribute is missing, the node's documented default is used and labelled as a default (*the default of 1 retry*). Durations may be converted to a more natural unit (`delay_msec="1000"` as *1 second*).